Milestone: Application in Python

Group 4

Hitaxi Pareshkumar Lethwala

Dev Mittalkumar Patel

IE 6700 Data Management for Analytics

Phone Student 1: 781-600-9414

Phone Student 2: 857-351-3855

lethwala.h@northeastern.edu

patel.devmi@northeastern.edu

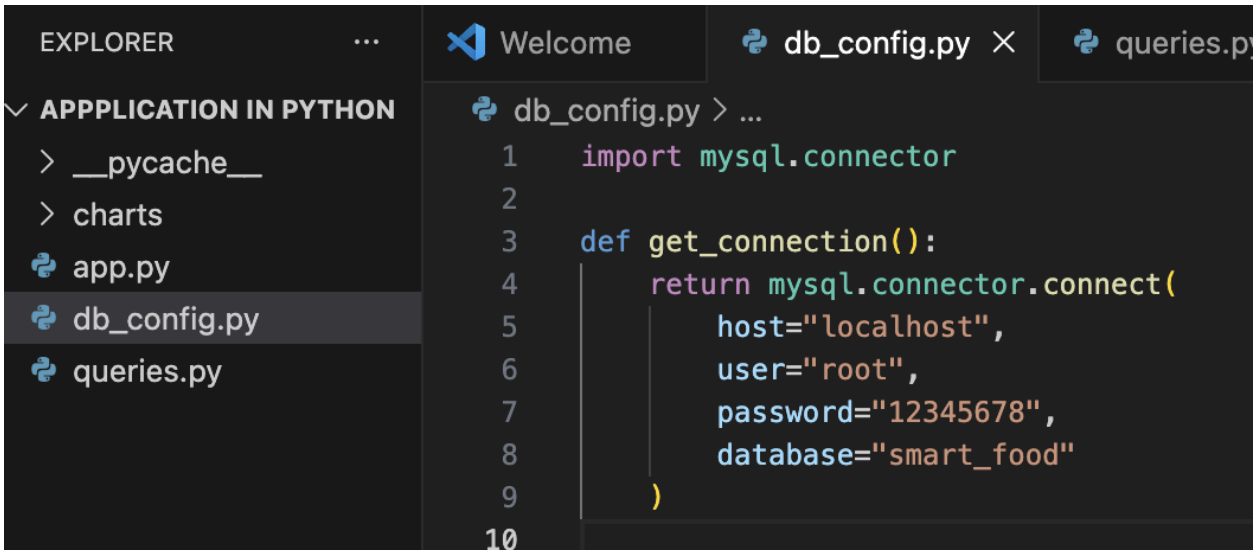
Percentage of Effort

Contributed by Student1: 50%
Contributed by Student2: 50%

Signature of Student 1: Hitaxi. P. Lethwala
Signature of Student 2: Dev. M. Patel

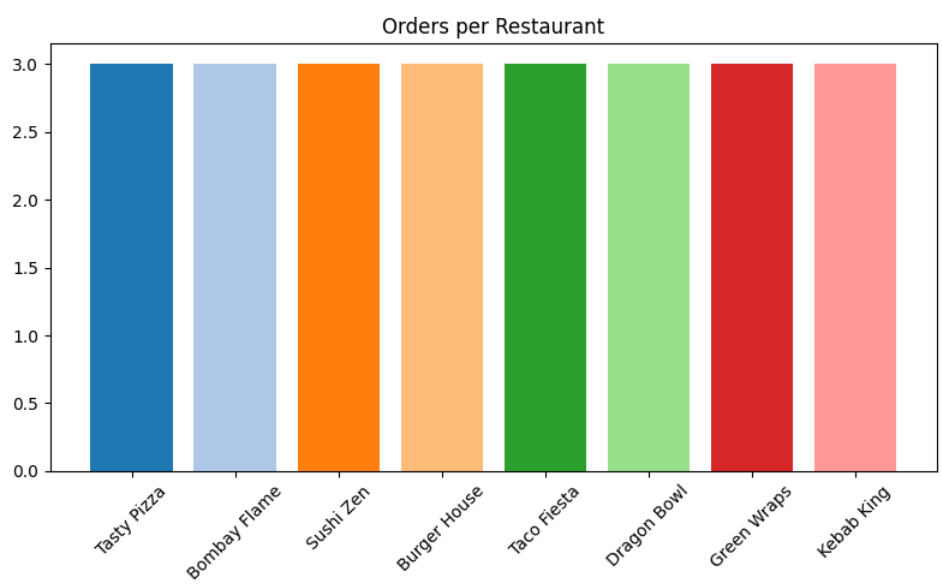
Submission Date: November 22, 2025

Code to connect MYSQL Database in Python(VS-code)



QUERY 1 — Orders per Restaurant

```
SELECT r.name AS restaurant, COUNT(o.order_id) AS total_orders
FROM orders o
JOIN restaurants r ON o.restaurant_id = r.restaurant_id
GROUP BY r.name;
def q1_orders_per_restaurant():
    return run_query("""
        SELECT r.name AS restaurant, COUNT(o.order_id) AS total_orders
        FROM orders o
        JOIN restaurants r ON o.restaurant_id = r.restaurant_id
        GROUP BY r.name;
    """)
df1 = q1_orders_per_restaurant()
plt.figure(figsize=(8,5))
plt.bar(df1['restaurant'], df1['total_orders'], color=colors2)
plt.title("Orders per Restaurant")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("charts/01_orders_per_restaurant.png")
plt.close()
```

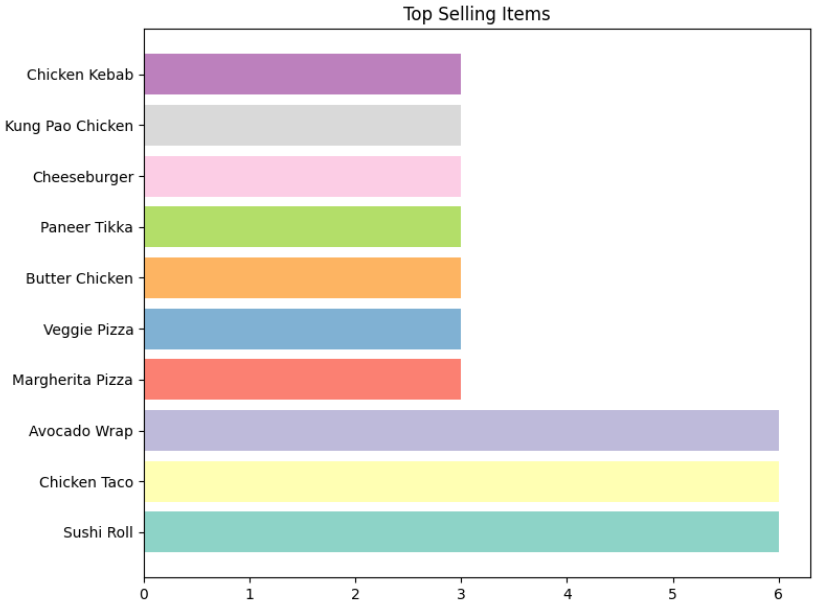


SIMPLE ANALYTICS

- Shows which restaurant handled the most customer orders.
- Helps identify high-demand restaurants.
- Useful for staffing, promotions, and prioritizing partnerships.

QUERY 2 — Top Selling Menu Items

```
SELECT m.name, SUM(oi.quantity) AS qty
FROM menu_item m
JOIN order_item oi ON m.item_id = oi.item_id
GROUP BY m.name
ORDER BY qty DESC
LIMIT 10;
def q2_top_selling_items():
    return run_query("""
        SELECT m.name, SUM(oi.quantity) AS qty
        FROM menu_item m
        JOIN order_item oi ON m.item_id = oi.item_id
        GROUP BY m.name
        ORDER BY qty DESC
        LIMIT 10;
    """)
df2 = q2_top_selling_items()
plt.figure(figsize=(8,6))
plt.barh(df2['name'], df2['qty'], color=colors1)
plt.title("Top Selling Items")
plt.tight_layout()
plt.savefig("charts/02_top_selling_items.png")
plt.close()
```

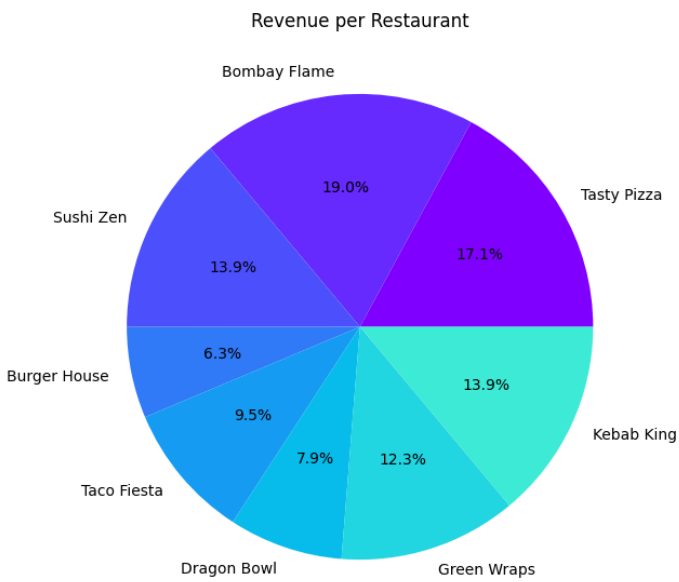


SIMPLE ANALYTICS

- Identifies the most frequently ordered items.
- Helps restaurants decide which items to promote or keep stocked.
- Useful for improving menu design & inventory planning.

QUERY 3 — Revenue per Restaurant

```
SELECT r.name AS restaurant, SUM(o.total_amount) AS revenue
FROM orders o
JOIN restaurants r ON o.restaurant_id = r.restaurant_id
GROUP BY r.name;
def q3_revenue_per_restaurant():
    return run_query("""
        SELECT r.name AS restaurant, SUM(o.total_amount) AS revenue
        FROM orders o
        JOIN restaurants r ON o.restaurant_id = r.restaurant_id
        GROUP BY r.name;
    """)
df3 = q3_revenue_per_restaurant()
plt.figure(figsize=(7,7))
plt.pie(df3['revenue'], labels=df3['restaurant'], autopct='%1.1f%%', colors=colors3)
plt.title("Revenue per Restaurant")
plt.savefig("charts/03_revenue_distribution.png")
plt.close()
```

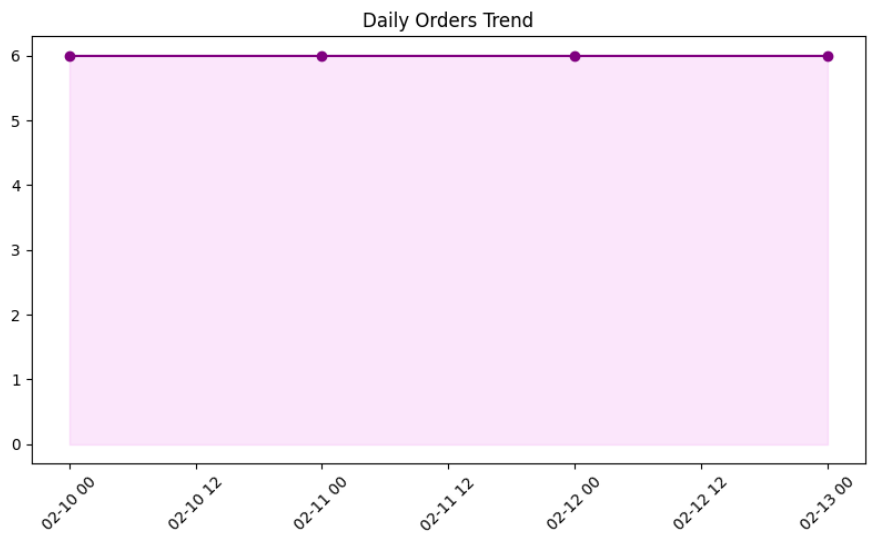


SIMPLE ANALYTICS

- Shows which restaurants generate highest earnings.
- Useful to identify top revenue partners.
- Helps company negotiate commission percentages more effectively.

QUERY 4 — Daily Order Trend

```
SELECT DATE(order_time) AS day, COUNT(order_id) AS total_orders
FROM orders
GROUP BY DATE(order_time)
ORDER BY day;
def q4_daily_orders():
    return run_query("""
        SELECT DATE(order_time) AS day, COUNT(order_id) AS total_orders
        FROM orders
        GROUP BY DATE(order_time)
        ORDER BY day;
    """)
df4 = q4_daily_orders()
plt.figure(figsize=(8,5))
plt.plot(df4['day'], df4['total_orders'], marker='o', color="purple")
plt.fill_between(df4['day'], df4['total_orders'], alpha=0.2, color="violet")
plt.xticks(rotation=45)
plt.title("Daily Orders Trend")
plt.tight_layout()
plt.savefig("charts/04_daily_orders_trend.png")
plt.close()
```



SIMPLE ANALYTICS

- Shows daily fluctuations in customer demand.
- Helps predict busy vs. slow days.
- Useful for staffing, delivery driver scheduling, and supply management.

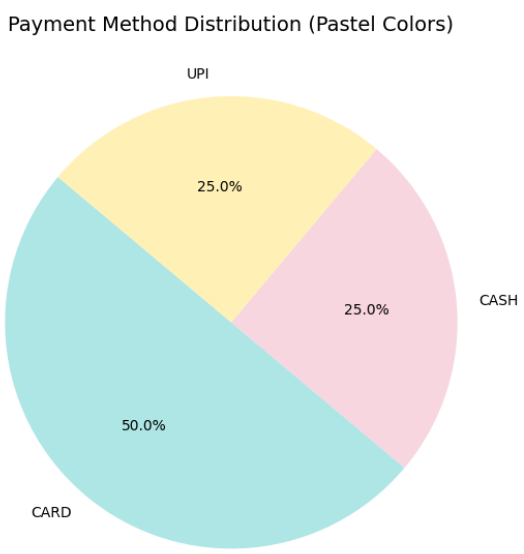
QUERY 5 — Payment Method Distribution

```
SELECT payment_method, COUNT(*) AS usage_count
FROM orders
GROUP BY payment_method;
def q5_payment_methods():
    return run_query("""
        SELECT payment_method, COUNT(*) AS usage_count
        FROM orders
        GROUP BY payment_method;
    """)
df5 = q5_payment_methods()

count_col = [col for col in df5.columns if df5[col].dtype != object][0]
method_col = [col for col in df5.columns if df5[col].dtype == object][0]

pastel_colors = ["■#AEE6E6", "■#F7D6E0", "■#FFF1B5"]

plt.figure(figsize=(7, 6))
plt.pie(
    df5[count_col],
    labels=df5[method_col],
    autopct='%1.1f%%',
    colors=pastel_colors,
    startangle=140
)
plt.title("Payment Method Distribution", fontsize=14)
plt.tight_layout()
plt.savefig("charts/05_payment_methods_pastel.png")
plt.close()
```



SIMPLE ANALYTICS

- Shows which payment methods customers prefer (cash, card, wallet).
- Helps company decide:
    - Whether to promote digital payments
    - Whether to offer discounts for card/UPI
    - Whether to support new payment methods

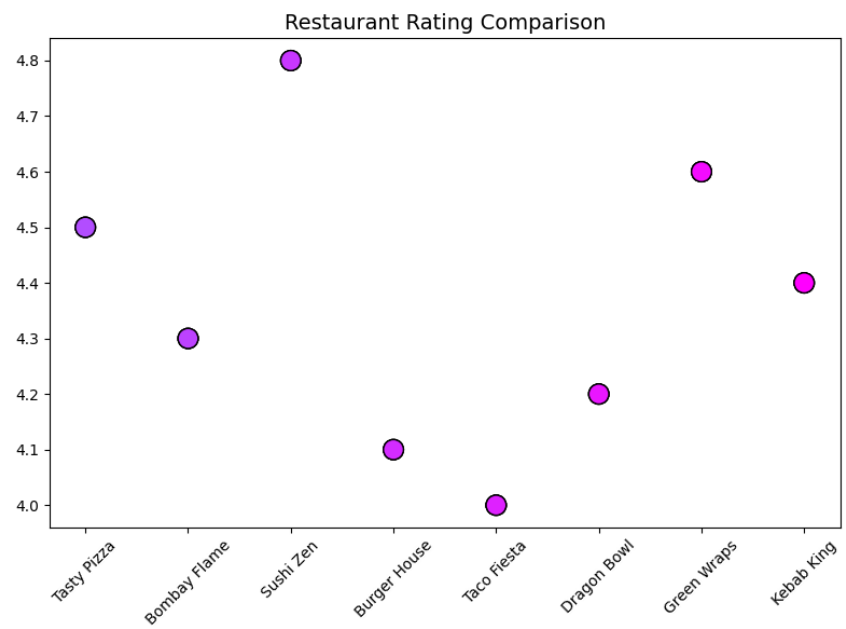
QUERY 6 — Restaurant Ratings

```
SELECT name AS restaurant, rating
FROM restaurants;
def q6_restaurant_ratings():
    return run_query("""
        SELECT name AS restaurant, rating
        FROM restaurants;
    """)
df6 = q6_restaurant_ratings()

x_col = [col for col in df6.columns if col != 'rating'][0]

num_points = len(df6)
point_colors = plt.cm.cool(np.linspace(0, 1, num_points))

plt.figure(figsize=(8, 6))
plt.scatter(df6[x_col], df6['rating'], s=180, c=point_colors, edgecolors='black')
plt.xticks(rotation=45)
plt.title("Restaurant Rating Comparison", fontsize=14)
plt.tight_layout()
plt.savefig("charts/06_ratings_scatter.png")
plt.close()
```



SIMPLE ANALYTICS

- Allows comparison of restaurant quality.
- Helps company identify:
  - Low-rated restaurants (need improvement)
  - High-rated ones (promote more)
  - Customer satisfaction trends

ANALYSIS 1 - Restaurant Business Performance

(Based on Query 1: Orders per Restaurant + Query 3: Revenue per Restaurant + Query 6: Ratings)

- Restaurants that consistently receive high order volume also show strong revenue performance, indicating stable customer preference.
- Higher-rated restaurants tend to attract more customers and generate more income, reinforcing that quality drives demand.
- Restaurants with low orders, low revenue, or low ratings may require intervention such as improved menu quality, faster delivery, or promotional support.
- This combined analysis helps the platform identify top-performing restaurants, emerging partners, and underperforming vendors that need improvement.

ANALYSIS 2 - Menu Item Trends & Customer Demand

(Based on Query 2: Top Selling Items + Query 4: Daily Orders Trend)

- Top-selling menu items significantly impact overall daily order trends, showing which food categories drive customer demand.
- Daily order patterns reveal peak demand periods, helping restaurants stock the right ingredients for top-selling items.
- Items that consistently appear at the top of sales charts should be promoted, featured in combos, or bundled with offers.
- Low-selling items may not match customer taste preferences and can be re-priced, redesigned, or replaced to reduce waste.

ANALYSIS 3 - Customer Behavior & Payment Preferences

(Based on Query 5: Payment Method Distribution + Query 4: Daily Orders)

- Payment method trends indicate whether customers prefer digital payments (UPI/Card) or traditional cash-on-delivery.
- If digital methods dominate, loyalty programs or cashback offers could further increase customer retention.
- Daily ordering behavior paired with payment preference reveals how customers behave at different times (e.g., more UPI on weekdays, more cash on weekends).
- This helps optimize payment gateway partnerships, delivery workflow, and checkout user experience.

ANALYSIS 4 - Overall Platform Performance

(Based on All 6 Queries)

- Order volume, revenue distribution, and ratings together highlight the overall health of the food delivery platform.
- Restaurants with high ratings, strong sales, and consistent daily orders should be prioritized for featured placements in the app.
- Payment and ordering patterns reflect the convenience and trust customers have in the platform.
- Combining demand analytics, restaurant performance, rating scores, and payment behavior enables data-driven decisions for:
    - improving customer satisfaction,
    - refining menu and pricing strategy,
    - optimizing delivery schedules,
    - supporting growth in high-potential areas.